

M-Agent 当前项目进度与设计计划

蓝色章节条用于定位结构；黄色提示框表示结论、边界或风险；表格中的加粗项是当前决策重点。

源文档：current-project-progress-and-design-plan.md · 状态日期：2026-07-30

Current Project Progress and Design Plan

状态快照：2026-07-29 (**P8 灰度与 LangGraph 完整矩阵已完成**：think_life_v1 与 langgraph_v1 均为 38/38 variant 可执行；共享验收矩阵 56/56 全绿；持久 SQLite checkpointer、SP/AT/Scene 接入与 runtime_engine 灰度路由已落地)

范围：三大结构性功能的语义基线、现有 ThinkLife Runtime 整理，以及后续 LangGraph Runtime 迁移。

本文回答两个问题：

1. 当前项目实际上已经完成了什么、还缺什么；
2. 应按照什么依赖顺序继续设计、实现和验收。

本计划遵循一个核心原则：

先冻结设计总线，再冻结跨 Runtime 的共享语义测试；先让测试准确表达稳定的领域行为，再按照 TX → SP → AT 的实现依赖完成当前 Runtime 基线，之后才开始 LangGraph PoC。旧实现不是新实现的真理，设计总线和共享语义测试才是新旧 Runtime 的共同验收标准。

1. 当前总体结论

当前结论：P0 ~ P8 已完成。双 Runtime 共享矩阵 38/38 variant 可执行且 56/56 验收用例全绿 (think_life_v1 48 + langgraph_v1 56 含 runtime 选择器覆盖)。 langgraph_v1 已接入持久 SQLite checkpointer、SP/AT 共享外层、runtime_engine 固定归属与可回退的默认路由。下一步进入稳定观察期，再逐步把新建 transaction 默认路由切到 LangGraph 并安全收缩旧 transaction 循环。**P5/P6/P7/P8 不包含 M-Agent-UI 或 M-Agent-Desktop 的界面改造。**

具体来说：

- 三大结构性功能及其边界已经明确：
- TX：Transaction 事务线与 Scene 情景线；
- SP：Conversation 内的感知刺激池；
- AT：无来源刺激的 Transaction 归因。
- 当前版本的设计总线已经按最新决策更新，见 [overall-design-architecture.md](#)。

- 三大功能的详细语义测试规格已经同步当前版决策，见 [runtime-migration-semantic-test-spec.zh-CN.md](#)。
- LangGraph 迁移计划已经统一 Transaction Store/checkpoint 权威边界和 P0 ~ P8 阶段入口，见 [langgraph-runtime-migration-plan.zh-CN.md](#)。
- 验收平台已经完成 Scenario → Variant → Runtime binding 合同、28 个 TX/SP/AT 顶层场景、27 个 Core、7 个必测 Robustness 和 1 个 Matcher Evaluation variant。
- 共享 Runtime Harness、ThinkLife adapter、CLI、Web 分层工作区、结构化报告和 P1 分层差距表均已完成。
- think_life_v1 的 35 个当前必测 variant 已全部可执行且全绿。P7 收口后结果为 35 Passed、0 Registered Known Gap、0 Unexpected；P1 时的 “35 个 Known Gap” 只保留为 pre-P2 历史基线。
- AT-10 已冻结 30 条 matcher 评测数据，Harness evaluate_matcher 已接入确定性 offline_policy_v1，并达到 ≥90% 总准确率 / ≥85% reuse recall 冻结阈值；真实 LLM matcher 仍独立于确定性 Core Gate。详细规格第 8 节的 25 条 Robustness requirement 已全部映射到 7 个必测 Robustness variant。
- langgraph_v1 已在 P8 完成完整矩阵：38/38 variant 可执行，共享验收 56/56 全绿。实现位于 src/m_agent/runtime/langgraph/ 与 langgraph_v1 acceptance adapter；TX-01 保留 graph 原生切片，TX-02 ~ TX-10 与全部 SP/AT 通过共享 Store 外层与 ThinkLife 等价 handler 驱动；持久 checkpoint 使用 langgraph-checkpoint-sqlite；灰度路由见 src/m_agent/runtime/routing.py 与 M_AGENT_DEFAULT_RUNTIME_ENGINE 环境变量。
- ThinkLife 已具备 P2 的四态 Transaction、activation/delegate、revision/CAS、transition ledger、SQLite 参考 Store、transaction 级 Pause/Delete/Restore、Scene append 幂等及原子 Flush 基础，并已接入 P3 Store-backed Stimulus Pool、P4 来源分流 / pause+未 flush complete 候选 / fake matcher 安全新建 / match→restore activation，以及 P5 的 SP → AT → TX 端到端收口 (drainer lost-wakeup 闭合、lease takeover fencing、Schedule claim 可观察、Feedback consume-once + disposition、Matcher 阈值达标)。
- ThinkLife 已在 P7 接入 durable effect ledger、Feedback ingress outbox、capability delivery guarantee 声明、Schedule delivery 幂等与 UI Pause 下的 schedule run 控制恢复；EffectCoordinator 与扩展后的 TransactionScheduleCoordinator 提供与 FlushCoordinator 同级的故障注入与重启恢复观察面。
- LangGraph Runtime 已在 P8 完成完整矩阵：持久 SQLite checkpointer、SP/AT/Scene 共享外层接入、runtime_engine 灰度路由与双 Runtime 38/38 可执行；旧 Runtime 收缩进入稳定观察期。

2. 当前项目进度

2.1 进度总表

工作项	当前状态	已有产出	尚未完成
三大功能设计总线	当前版已确认	三大功能的意义、边界、领域逻辑及最新人机控制语义	后续版本只在领域规则实际变化时修订
目标语义测试规格	当前版已同步并已转成 P1 可执行基线	TX-01~10、SP-01~08、AT-01~10，含 activation、UI Pause/Delete、Feedback Gate、异步工具和一次性 Plan 子场景	双 Runtime 38/38 可执行、56/56 验收全绿
详细设计决策	首版方案已进入可执行契约	优先级、持久化、Matcher v1、合成数据、flush 原子边界、执行批次校验	按 P1 证据在 Runtime 实现中验证和收敛
P1 共享测试合同	已完成并满足退出条件	Scenario/Variant/Runtime schema; 28 个场景、27 Core、7 Robustness、1 Matcher Evaluation、3 PoC; 共享 Harness; 双 Runtime 38/38 可执行; CLI、Web、报告与分层差距表	合同随稳定领域语义维护
旧语义目录	可执行但需要降级定位	INV-01~INV-14，Gate 20 项执行用例，Full 36 项执行用例	作为历史映射和 Supporting Tests，不再作为目标 Core Gate
ThinkLife Runtime	P7 已完成	四态与 tombstone; activation/delegate; revision/CAS; 幂等 ledger; SQLite Transaction/Scene/Flush; Pause/Delete/Restore; Store-backed Stimulus Pool; Schedule delivery 幂等; effect ledger + Feedback outbox; capability delivery guarantee; 35/35 全绿	稳定观察期后配合默认路由收缩
跨 Runtime Harness	P8 已完成	共用参考 Store; 双 Runtime 完整矩阵; load_effects / load_schedule 故障注入; process 重建与 checkpoint 持久化	生产 Chat API 按 runtime_engine 分发
LangGraph Runtime	P8 已完成	持久 SQLite checkpointer; SP/AT/Scene 共享外层; runtime_engine 路由; 38/38 variant 可执行	默认路由切 LangGraph; 稳定观察后删除旧循环

2.2 已完成的领域澄清

Transaction 与 Scene

已经确认：

- Transaction 是稳定、持久、可能跨多轮的单任务交互记录。
- Transaction 的内容表现为 WM，状态表现为任务状态。
- transaction_id 表示长期稳定的任务身份；activation_id 表示该任务的一次有效执行批次；delegate_id 表示该批次中的一次工具委托。

- 目标状态只有：
- `continue` ；
- `pause` ；
- `complete` ；
- `archive` 。
- 模型不能直接选择 `archive` ；只有 flush 可以执行 `complete → archive` 。
- `pause` 和未 flush 的 `complete` 可以被无来源刺激重新匹配。
- `archive` 不参加自动匹配，但允许通过合法显式 transaction ID 或有效预登记 schedule run 恢复。
- Flush 是人工或自动触发、但始终用户可见的会话语义边界；Flush 后 `archive` 不再参加自然续接属于预期结果。
- 当前版本只处理一次性 Scheduled Plan：安排时保存原 transaction ID，等待期间保持 `pause`，到期时为原 transaction 开启新 activation，执行完成进入 `complete`，之后才由 Flush 归档。
- UI Pause 是 transaction 级直接控制动作：中止当前思考，当前刺激不重新入池，transaction 进入人工 `pause`，当前 activation 及其未决 Feedback 永久失效。
- UI Delete 终止该 transaction 的生命周期，使其不再参加匹配或恢复，后续关联 Feedback 永久拒绝。
- 每个 conversation 只有一条连续 Scene；多个 transaction 的活动按实际发生顺序进入同一条 Scene。

感知刺激池

已经确认：

- 刺激池的优先级只在单个 conversation 内生效。
- 未来 Scheduled Plan 在到期前不属于当前刺激池。
- 刺激池能保存多个类别的刺激。
- 高优先级先消费；同优先级按接纳顺序 FIFO。
- 同一 conversation 始终逐个消费。
- 单消费者串行处理的是一次刺激对应的 Thinking 与状态提交，不等待异步工具执行完成。
- Thinking 发出工具委托后，本次刺激即可消费结束；工具结果以后续 Feedback 刺激重新申请进入感知层。
- Feedback 在入池前完成执行批次和委托有效性校验；预期废弃的 Feedback 不进入正常刺激池。
- UI Pause/Delete 是直接控制动作，不作为普通刺激排队。
- 刺激池只负责接纳与排序，不负责 transaction 归因。
- 已归因刺激即使重新入池，也必须保持原刺激身份和 transaction 归属。

Transaction 归因

已经确认：

- 是否有明确 transaction 来源，是归因流程的第一分支；不能只按刺激类型判断。
- 有明确 transaction ID 的刺激绕过语义 Matcher，但仍必须通过该来源类型的确定性有效性校验。
- Feedback 必须同时匹配 `transaction_id + activation_id + delegate_id`；旧 activation、无效 delegate、人工暂停或已删除 transaction 的 Feedback 在感知入口废弃。
- Scheduled Plan 和合法的显式恢复动作可以按照各自规则为原 transaction 开启新的 activation；旧 Feedback 不能充当恢复动作。
- 指定 transaction 在活动记录和归档记录中都不存在时，应明确拒绝，不能偷偷新建。

- 无来源刺激只允许在同一 conversation 的 `pause` 和未 flush 的 `complete` 中匹配。
- 明确匹配唯一候选时复用原 transaction，开启新的 activation，并恢复为 `continue`。人工暂停后的 transaction 仍可由无来源刺激正常匹配；Feedback 拒绝规则不阻止该路径。
- 无匹配时新建。
- 匹配模糊时新建。
- 一旦归因，后续执行、重入、恢复和 Scene 记录都必须保持 transaction 归属；这不代表已经失效的 activation 可以复用。
- 当前版本支持合法显式 transaction ID 或有效预登记 schedule run 恢复 archive；通过自然语言询问主题、检索历史 transaction 再恢复留待后续版本。
- 周期性 Scheduled Plan 及其重复激活状态线留待后续版本。

2.3 已形成的首版详细设计

详细方案集中记录在 [runtime-migration-semantic-test-spec.zh-CN.md](#) 的第 10 节。目前已经形成以下首版决策：

- 执行身份与人机控制：
- transaction 身份跨多轮和多次恢复保持稳定；
- 每次独立激活使用新的 activation 身份；
- 工具委托及其 Feedback 只能在所属 activation 内有效；
- UI Pause/Delete 使当前 activation 失效；
- Transaction 恢复后旧 activation 的晚到 Feedback 仍然拒绝；
- `pause_reason` 至少区分 `awaiting_feedback/awaiting_user/scheduled_wait/manual_hold`；
- 无来源刺激匹配 `pause(awaiting_feedback)` 时，原子失效旧批次再开启新批次。
- 优先级：
- 数值越小，优先级越高；
- 在 conversation 内排序；
- 相同优先级以接纳时产生的稳定序号保证 FIFO；
- 刺激进入池时冻结本次有效优先级；
- 重入保持原刺激身份、transaction ID、优先级和接纳序号。
- 持久化：
- 领域上分离 Transaction、Stimulus Inbox、Scene、Schedule、Effect Ledger 和 Flush Journal；
- 首个参考后端使用 SQLite；
- 统一 UoW 协调 Transaction、Inbox、Schedule、Scene 与 effect/outbox 的相关 mutation；transition ledger 保存 command digest 和完整结果引用；
- stimulus 的 ingress key、accepted_seq、claim 和 disposition 是可恢复领域事实；conversation owner/lease 与 consumer/claim epoch 是 fencing token，旧 claimant 的新提交必须得到 `StaleClaim`；
- transition replay 先查 ledger：已提交的同 ID/digest 返回原结果，未提交的新命令才校验 fencing/revision；
- LangGraph checkpoint 只负责图恢复，不作为 Transaction 或 Scene 的权威来源。

- 一次性 Schedule:
- 使用稳定 `schedule_id + schedule_run_id` 去重;
- 每次投递使用稳定 delivery ID/generation, 并保存 claimed activation;
- 已有有效 activation 时进入 `blocked_on_activation`, 不并行开批次;
- `manual_hold` 不被自动绕过, deleted transaction 不被复活;
- claimed run 遇到 UI Pause 时, 仍在 Thinking 的 delivery aborted、已经 consumed 的 delivery 保持不变, run blocked; 遇到 UI Delete 时 run cancelled; 恢复后只在安全边界生成下一代 delivery;
- claimed schedule activation 被新的合法 activation 取代时, run 解除旧绑定并以 `superseded_by_new_activation` 阻塞, 不能悬挂或并行执行;
- schedule execution 进入 `pause(awaiting_user)` 时, run 保持 claimed 但解绑已结束 activation; 用户协作恢复时再原子绑定新 activation;
- claimed run 内为同一 transaction 链式登记下一 run 属于 Future, 当前版本明确拒绝;
- 同一 transaction 同时最多一个 claimed run; Schedule preconsume 未开启 activation 时, delivery 必须以 `handled` 终结, 不能悬挂在 claimed;
- 有效预登记 run 可以按明确来源恢复 complete/archive。
- 异步 effect:
- result 与 Feedback ingress outbox 同提交, Inbox 以唯一 ingress key 去重;
- accepted、idempotent existing 和 admission Expected Discard 都 terminal ack outbox; 只有基础设施错误重试;
- `idempotent` 使用同 key 重试至确认, 允许多 attempt 但只产生一次可见效果;
- `at_most_once` 最多 dispatch 一次, 失联不重试并暴露 `uncertain` ;
- `at_least_once` 使用同 key 重试至 destination 最终恢复可达并确认, 在最终可达前提下保证至少一次并允许重复; 当前版本不定义独立的 effect cancel, UI Pause/Delete 也不等价于撤销外部 effect。
- Matcher v1:
- Runtime 先确定合法候选集合;
- Matcher 只能在合法候选中选择, 或者决定新建;
- 模糊、异常、超时或非法输出统一安全降级为新建;
- 确定性 Core Gate 使用 fake matcher;
- 真实 matcher 使用独立合成数据集评估, 不把模型波动混入结构测试。
- Flush:
- Flush 是用户可见的会话语义边界, 不是不可见的后台清理;
- Scene 处理边界与 `eligible complete → archive` 构成一次逻辑提交;
- 提交前失败应整体回滚;
- 提交后外部物化失败应能够根据持久记录继续, 不能重复归档或丢失 Scene 内容。

执行批次因果关系、UI 控制与 Flush 可见性属于稳定领域逻辑, 已经进入 [overall-design-architecture.md](#)。具体优先级数字、存储后端、Matcher 数据结构和 outbox 等仍属于可演进方案, 不进入设计总线。

上述 activation、UI Pause/Delete、一次性 Scheduled Plan 和异步工具消费边界已经同步到详细测试规格。P1 已将完整目标清单转换为 Runtime 无关的可执行契约, 并通过统一 Harness 对 ThinkLife 的全部 35 个当前必测

variant 形成证据；下一步由 P2 按这些证据实现 TX Foundation。

2.4 现有验收平台状态

当前工作区已经具备：

- pytest 隔离执行 Runner；
- P1 Scenario/Variant/Runtime binding 数据模型和完整清单；
- CLI：
- 查看 P1 合同与覆盖；
- 按 Scenario、Runtime 和层级运行 P1 场景；
- 查看目录；
- 运行 Gate；
- 运行 Semantic Full；
- 查看历史报告；
- 本地 UI：
- 默认页只展示 P1，共享合同与旧 INV-01~INV-14 Supporting Tests 不再混排；
- 运行总览、TX/SP/AT 领域统计、Variant 索引与单项证据主从视图；
- 按领域、层级、状态和关键字筛选；
- 在未满足检查中直接对照 Expected、Actual、Evidence 和 Known Gap key；
- Passed、Registered Known Gap、Unexpected Failure 采用独立视觉状态；
- 按 Runtime 隔离并恢复最近一次运行结果；
- 旧 Gate/Full 界面降级到 /legacy 历史兼容入口；
- 启动后显示访问地址；
- 结构化运行报告；
- normalized semantic trace；
- Known Gap 与新增失败的差异化展示；
- 30 条 AT-10 matcher 数据和独立 evaluator；
- 25 条详细 Robustness requirement 到 7 个必测 variant 的 manifest；
- [ThinkLife P1 分层差距表](#)。

P1 合同清单保持不变；下表是 **2026-07-29 P3 后复验**，不是 2026-07-28 的 pre-P2 历史快照：

层级	规格清单	ThinkLife 可执行	Passed	Registered Known Gap	LangGraph
Core	27/27	27/27	17	10	27 Not Implemented
Robustness	7/7	7/7	3	4	7 Not Implemented
Matcher Evaluation	1/1	1/1	0	1	1 Not Implemented
合计	35/35	35/35	20	15	35 Not Implemented

ThinkLife 全矩阵能够真实执行当前 Transaction、Stimulus Inbox、Attributor、Scene 等组件，并把每个目标断言归一化为 check 与 facts。P3 复验后，TX-02/core、TX-05/core、

TX-08/core、SP-01/core、SP-01/durable_ingress_restart、SP-02/core、SP-03/core、SP-04/core、SP-05/core、SP-07/core、SP-08/core 的 stale Known Gap 已清理并转为 Passed。其余 15 个差距仍由唯一 check key 登记，未登记失败为 0。AT-10 使用冻结的 30 条数据独立评测，Robustness manifest 的 25 条要求也全部有明确 variant 承接。

截至本状态快照，旧目录的最新本地结果为：

Profile	语义项	执行用例	Passed	XFailed	结果
Gate	14	20	14	6	known_gaps
Semantic Full	14	36	30	6	known_gaps

按语义项统计，旧 Gate 中有 8 项通过、6 项为 Known Gap。

这组旧结果只说明 Supporting 目录 INV-01~INV-14 的现状，不能替代新的共享 Scenario matrix，也不能替代独立 P2 Foundation Gate。P1 的 35 Known Gap 快照已冻结为历史证据；当前结果以本节 post-P2 表和结构化报告为准。

2.5 旧 Supporting 目录的 Known Gap（剩余 2 个）

当前验收目录仍显式记录以下已知差异：

旧编号	当前差异	在新结构中的定位
INV-05	flush 仍依赖 legacy Thinking state，未完整读取 TransactionRecord 中的待处理内容	TX
INV-10	reply 的工具审计记录不完整	TX 下层 Robustness 或 Supporting Test

原先并列的另外 4 个已在 P2 ~ P8 的 TX/SP/AT 收口中关闭，对应 Supporting Test 已改为断言新契约而非 xfail：

旧编号	原差异	关闭方式
INV-01	用户消息可能先写 Scene，再完成 inbox admission	Sourceless 用户话语的 Scene 写入移到归因之后（Gateway 只写已带 transaction_id 的刺激），入队不再被 Scene 失败阻塞
INV-02	worker 空队列检查与退出之间存在 lost-wakeup 窗口	Drainer 交接补齐；INV-02 用例不再触发降级分支
INV-12	重入刺激虽保留 transaction ID，但归因可能忽略它并新建 transaction	显式 source 走 _resolve_explicit_source，锁定原事务线并按需 restore
INV-13	重复 Feedback 尚无稳定的持久消费键	activation + delegate 构成持久消费键：重复 Feedback 按同一 ingress 身份幂等吸收，delegate 只消费一次

这些是旧 Supporting 目录已经发现的差异，不是新目标语义的完整缺口清单。新的 27 个确定性 Core Gate、7 个必测 Robustness variant 和 AT-10 Matcher Evaluation 已经

全部在 ThinkLife Runtime 上执行；完整结果以
[P1 分层差距表](#)为准。

2.6 当前 ThinkLife Runtime：P3 已接入后的剩余差距

TX: Transaction 与 Scene

P2 已完成：

- `continue/pause/complete/archive` 是权威四态；legacy scheduler status 只作为兼容投影；
- `lifecycle_status=deleted` 是独立永久 tombstone，正常 archive 只能由 Flush 产生；
- 每个 transaction 持久保存 `revision`、`runtime_engine`、schema version、WM 和任务状态；
- activation 与 delegate 有独立持久身份，Store 约束同一 transaction 只能有一个 active activation、一个 pending delegate，Pause/Delete 会永久失效旧批次；
- transaction-side UoW 使用 CAS/fencing 和 transition ledger；相同 transition ID 与 canonical command 重放第一次 JSON 归一结果，不同 command 明确冲突；
- 已知 transaction ID 下的 Pause、Complete、Delete、Restore 和 archive→continue 均使用真实领域状态机；Restore 总是产生新 activation；
- Transaction、Scene、watermark、Flush journal/outbox 共用一个 SQLite 参考 Store；Scene 使用稳定 `append_id`，Flush 在同一事务内推进 watermark 并只归档 eligible complete transaction；
- Flush ID、activation/delegate 归属、不可变 transaction 归属和 revision 单步推进均有 Store 级不变量保护；commit 后物化窗口可在重启后恢复。

P3 复验后仍未完成：

- M-Agent-UI 与 M-Agent-Desktop 的 Pause/Delete/Restore 控件和交互接线；
- in-flight worker latch 与当前 stimulus 的持久 abort/no-reentry disposition 仍需在完整 drainer 路径中收口；
- P4 的来源分流、`pause + 未 flush complete` 候选集合和 matcher 端到端续接；
- P7 的生产级跨 Store UoW、真实 effect/result outbox、外部记忆物化与进程级故障强度。

当前实现证据可见：

- [contracts.py](#)
- [transaction_registry.py](#)
- [runtime.py](#)
- [jsonl_store.py](#)

SP: Conversation 内刺激池

当前已有：

- Stimulus 数据结构；
- 优先级队列；
- 不同刺激入口；
- 按 thread 组织的消费循环。

主要差距：

- StimulusInbox 已接入 Store-backed partition/ready/claim/disposition; 生产 drainer 仍按 `thread_id` 唤醒, 但 claim/query 已能以 `conversation_id` 为边界观察和消费。
- 当前排序已采用 admission 时冻结的 effective priority 与 `accepted_seq`, 同优先级 FIFO 不再依赖 `occurred_at`。
- Inbox 已成为 stimulus ingress/accepted_seq/claim/disposition 的参考 SQLite 权威; P7 仍需补生产级跨 Store UoW、effect/result outbox 与更强进程恢复测试。
- 刺激类别默认优先级已区分 user/feedback/scheduled/observation。
- Gateway 已在 admission 前把无效 Feedback 写为 `expected_discard`, preconsume 失效也会持久化 stage/reason。
- 当前工具执行仍可发生在刺激处理调用栈内, 尚未证明 “Thinking 发出异步委托后立即释放消费者、工具结果以后续 Feedback 返回” 的目标边界。
- 当前没有 effect result→Feedback ingress outbox; 真实 capability 也尚未声明其 `idempotent / at_most_once / at_least_once` guarantee。
- P5 已闭合 drainer 空队列检查与 unregister 之间的 lost-wakeup 窗口; `SP-06/core` 与 `SP-01/lease_takeover_fencing` 已转绿。
- 新 SP 场景已经覆盖多生产者、单消费者、同优先级 FIFO 和 conversation 隔离; P5 后 SP Core 8/8 与 SP Robustness 2/2 均已通过。

当前实现证据可见:

- [inbox.py](#)
- [gateway.py](#)
- [drainer.py](#)

AT: 无来源刺激归因

当前已有:

- Execution Feedback 已强制校验 `transaction_id + activation_id + delegate_id`, admission 与 preconsume 对无效来源写 `expected_discard`; 成功 Feedback 会 consume-once, 并写入 durable disposition;
- 显式 `transaction_id` / Schedule 来源绕过 matcher, 并在 pause/complete 上执行 restore; 已登记 one-shot run 的 `scheduled_plan` 会原子 claim 并开启新 activation;
- 无来源候选精确为当前 conversation 的 `pause` 与未 flush `complete`;
- turn-local `candidate_N` fake matcher 契约, 异常/非法/模糊一律新建;
- 匹配 pause/complete 后复用原 ID 并开启新 activation;
- Harness `list_match_candidates`、`load_schedule` 与 `evaluate_matcher(offline_policy_v1)` 已接线;
- AT-01 ~ AT-09 Core Gate 全绿, `AT-10` 离线阈值已达标, `TX-03` 已收口。

主要差距:

- 真实 LLM matcher 仍独立于确定性 Core Gate, 不计入 M0 Runtime Gate。
- `TX-06` / `TX-07` 的生产级 Schedule/Inbox 半提交控制竞态、effect outbox 与 capability guarantee 明确归属 P7 Robustness。
- 刺激池 admission 排序本身能够保留 `priority_override=0`; 但当前用户/后台刺激归因路径在给 TransactionRecord 赋 priority 时使用布尔回退, 可能把合法的 0 替换成默认值。该差异属于归因后的

transaction priority 传播，不属于刺激池排序。

当前实现证据可见：

- [attributor.py](#)
- [core.py](#)
- [loop.py](#)
- [matcher_evaluation.py](#)

2.7 P2 完成后仍待实现的 Runtime 与迁移工作

P1 的共享场景、Harness、adapter、CLI/Web/report 和差距矩阵已经完成。以下缺口属于 Runtime 目标语义或后续迁移，不再属于“测试尚未接入”：

- 耐久 Stimulus Inbox、唯一 ingress、claim/disposition 已进入 P3；result→Feedback outbox 仍留待 P7；
- Schedule 与 transaction-side 绑定已经进入 P2；其与 P3 durable Inbox claim/disposition 的生产级统一提交仍待 P7 收口；
- Feedback admission Gate 和真实 Expected Discard 处理已接入 P3；
- Thinking 与异步工具执行解耦后的刺激消费链路；
- M-Agent-UI、M-Agent-Desktop 和 Chat API 的 transaction 控制面接线；
- LangGraph transaction graph；
- LangGraph 持久 checkpoint 与 resume；
- effect ledger 和节点重放幂等；
- 双 Runtime normalized trace 对比；
- 按新建 transaction 灰度选择 Runtime；
- 旧 Runtime 的安全收缩与退出。

2.8 已完成统一的状态权威与下游语义

旧版 LangGraph 迁移计划曾提出把 task state 和 WM 的权威状态迁入 checkpoint；该冲突现已在修订后的 [langgraph-runtime-migration-plan.zh-CN.md](#) 中消除。当前统一边界是：

- Transaction Store 保存稳定的 transaction 身份、WM 和任务状态；
- LangGraph checkpoint 保存恢复图执行所需要的运行状态；
- checkpoint 可以引用 transaction 版本，但不能静默覆盖领域状态；
- 两者之间的同步规则必须通过明确的 revision 和提交边界定义。

详细测试规格中“一次性 Scheduled Plan 执行后回到 pause”“只凭 transaction ID 接受 Feedback”等旧文字也已经删除，并纳入 activation、transaction 级 UI Pause/Delete、Feedback admission Gate 和异步工具消费边界。P0 的文档对齐项至此完成；后续若实现与这些语义不同，应记录为测试差距，而不是重新采用旧口径。

3. 测试平台的目标调整

3.1 保留基础设施，重做领域目录

现有 Runner、CLI、UI、报告和 trace 归一化可以继续使用，不需要推倒重写。

需要调整的是测试内容和顶层信息架构：

三大领域

└─ TX: Transaction 与 Scene

└─ SP: 感知层刺激池

└─ AT: Transaction 归因

每个领域下面再按测试层级展示：

领域

└─ Core Gate

└─ Robustness

└─ Matcher Evaluation (仅 AT)

└─ Supporting Tests

并且在每个场景下区分 Runtime：

同一语义场景

└─ think_life_v1

└─ langgraph_v1

3.2 目标测试目录

目标目录共 28 个顶层场景编号，但不能把它们全部合并成一个 Core Gate 通过率：

领域	顶层场景	确定性 Core Gate	独立评测	验证重点
TX	10	10	0	四状态、activation、UI Pause/Delete、一次性 schedule、feedback、多事务隔离、单 Scene、flush
SP	8	8	0	多类型、Feedback admission、优先级、FIFO、Thinking 逐个消费、异步工具解耦、并发与 conversation 隔离
AT	10	9	1	来源有效性、跨 activation 拒绝、候选边界、pause/complete 复用、新建、归属稳定、matcher
合计	28	27	1	AT-10 使用独立 Matcher Evaluation，不计入确定性 Runtime Gate

最新语义不需要增加 TX/SP/AT 顶层编号，但“挂在既有编号下”不等于“自动计入 Core”。

每个可执行 variant 还必须单独标记 layer；只有 layer=core 的 27 个顶层功能场景进入

Core 通过率，重启、重复投递、故障恢复和 lease takeover 等进入 Robustness。周期性 Schedule 和自然语言 archive 检索标记为 Future。

既有场景	Layer	P1 已纳入的 variant
TX-01	Core	新 transaction 首次运行产生 activation
TX-01	Robustness	UoW 已提交后重放返回第一次完整结果
TX-02	Core	UI Pause、中止当前刺激且不重入、旧 activation 失效；显式或无来源恢复均开新 activation
TX-03	Core	未 flush complete 可由无来源匹配或显式恢复重新激活
TX-05	Core	验收显式 ID 恢复 archive；TX-06 另验有效预登记 Schedule 恢复；自然语言检索为 Future
TX-06	Core	一次性 Schedule 正常完成；run/delivery、busy/manual/delete/supersede/awaiting_user、多 run 与 claimed 控制状态线
TX-06	Robustness	重复 generation、重启与 Schedule/Transaction/Inbox 半提交故障
TX-07	Core	Feedback 三元校验；admission 拒收与确定性 preconsume 失效；新 activation Feedback 可接纳
TX-07	Robustness	并发重复 Feedback、result→Feedback outbox 与三种 capability guarantee
TX-08	Core	Delete 直接中止、写 tombstone、永久拒绝恢复，且不影响其他 transaction
TX-10	Robustness	重复 Flush 与提交故障恢复
SP-01	Core	admission 废弃的 Feedback 不入池；确定性 preconsume 失效无副作用终止
SP-01	Robustness	durable ingress 重启、lease takeover 与 stale claimant fencing
SP-04	Core	Thinking 刺激逐个消费，但异步工具不占用消费者
AT-01/02/03/08	Core	来源有效性、deleted 候选排除、pause 新 activation 恢复及跨 activation 稳定性

既有编号及最新子场景已经统一记录在 [runtime-migration-semantic-test-spec.zh-CN.md](#)；P1 已把这份当前目标语义基线转换为共享可执行契约。

3.3 共享测试的证据原则

共享场景与断言不能依赖：

- 具体类名；
- 私有方法；
- 当前 ThinkLife 循环的内部步骤；
- LangGraph 节点数量或节点名称；
- Runtime 自己声称成功的日志。

主要证据应来自：

- 刺激池可观察快照及实际消费结果；

- Transaction Store 的状态、pause reason 与 lifecycle 快照；
- 当前 activation、有效 delegate 及失效批次快照；
- Stimulus Inbox 的 ingress key、accepted_seq、partition owner/lease、consumer/claim epoch、claim、StaleClaim 与最终 disposition；
- Feedback admission/preconsume 的接纳或预期废弃结果及 stage/reason；
- Schedule Store 的 schedule/run/delivery 身份、claimed activation、blocked reason、状态和目标绑定；
- Scene Store 的追加结果和顺序；
- flush 提交结果；
- effect result/Feedback ingress outbox、capability guarantee、attempts、ack/result、visible-effect count/range，以及 fake effect sink 中实际发生的外部效果；
- normalized semantic trace 中的因果顺序。

Trace 用来解释“为什么发生”，Store 和 effect sink 用来证明“实际上发生了什么”，避免 Runtime 通过自报日志形成自证循环。

3.4 结果状态

新平台至少应区分：

- Passed：实现满足该场景的目标语义。
- Known Gap：在运行前已经登记、当前实现明确尚未满足的目标语义。
- Failure：原本应通过，却出现未登记的错误或回归。
- Not Implemented：该 Runtime 或能力尚未实现，测试无法执行。
- Not Covered：目标语义尚无可执行场景，不能被计为通过。

其中，Known Gap 表示“已经知道实现与目标有差异”；Failure 表示“按当前承诺本应满足，却意外失败”。二者都不等于通过。

Expected Discard 不是新的测试运行状态，而是带 stage/reason 的场景证据。例如旧 activation Feedback 在 admission 被拒收，或者合法入池后因 UI 控制在 preconsume 失效，只要处置符合预期且没有领域副作用，场景仍是 Passed。前者从未获得 accepted_seq，后者此前已经合法参与排序；Scheduled Plan 的冲突使用 run 的 blocked/cancelled/idempotent disposition，不伪装成 Expected Discard。

旧 INV-01~INV-14 应保留为历史映射或 Supporting Tests，但不再作为 UI 第一层目录，也不再单独代表 Runtime 迁移的通过标准。

4. 实施依赖与阶段安排

4.1 运行数据流与实现顺序不同

系统实际运行时的数据流是：

SP 刺激池
→ AT 归因
→ TX Transaction 运行与 Scene 记录

但是实现依赖顺序应当是：

TX 基础
→ SP 刺激池
→ AT 归因
→ SP + AT + TX 端到端

原因：

- TX 先定义 transaction 身份、四状态、恢复方式、候选可见性和 Scene 关系，是 AT 的基础。
- SP 不需要依赖语义匹配，可以使用 fake consumer 独立完成。
- AT 同时依赖 SP 提供完整稳定的 stimulus，以及 TX 提供候选查询、创建、恢复和状态转换。
- 因此 AT 必须在 TX 和 SP 的基础语义稳定以后实施。

4.2 阶段总览

阶段	目标	主要产出	决策门
P0 (已完成)	冻结设计总线并 对齐下游文档	Overall Design Architecture、详细测试规格同步、状态 权威边界修订	三大功能边界获得确认，测试规格 和旧迁移计划不再与新语义冲突
P1 (已完成)	冻结跨 Runtime 测试契约	27 个共享 Core、7 个 Robustness variant/25 条 requirement manifest、30 条数据的 AT-10、共享 Harness、CLI/Web/report 和 ThinkLife adapter	35 个 ThinkLife variant 可执行且 无未登记失败；LangGraph 35 个 binding 明确 Not Implemented；已生成分层差距表
P2 (已完成)	完成 TX 基础	四状态、activation、UoW/revision、transaction 控制 原语、一次性 Schedule transaction-side 原语、持久 Transaction/Scene、flush 边界	独立 p2_foundation Gate 全 绿；6 个 TX variant 与 3 个直接受益的 AT variant 移除 Known Gap
P3 (已完成)	完成 SP	Store-backed conversation 池、durable ingress、 Feedback admission/preconsume、 claim/disposition、consumer epoch fencing、 Harness latch/disposition 观察面	SP Core 7/8 通过；SP-06 与 lease takeover 留作后续 robustness
P4 (已完成)	完成 AT	来源有效性、跨 activation 拒绝、候选过滤、fake matcher、match→restore、安全新建、 evaluate_matcher 离线评测入口	ThinkLife AT-01 ~ AT-09 Core Gate 全绿；TX-03 收口；AT-10 可报告指标但未达阈值

阶段	目标	主要产出	决策门
P5 (已完成)	完成 M0 基线	SP → AT → TX 完整链路; SP-06 lost-wakeup; lease takeover fencing; Schedule claim 可观察; Feedback consume-once + disposition; Matcher 阈值	ThinkLife 27 项 Core Gate 全绿; AT-10 独立达标; 剩余 Robustness 差距归属 P7 (已收口)
P6 (已完成)	LangGraph Transaction Graph PoC	单已归因 transaction、临时 checkpointer、可控 fake effect	双 Runtime 在所选共享切片上产生等价领域结果
P7 (已完成)	持久恢复与共享外层接入	effect ledger、Feedback outbox、capability guarantee、Schedule delivery 幂等与控制恢复	ThinkLife 35/35 全绿; 0 Registered Known Gap
P8 (已完成)	灰度与旧 Runtime 收缩	双 Runtime 完整矩阵、持久 LangGraph checkpoint、灰度路由	38/38 可执行、56/56 验收全绿; runtime_engine 固定归属与可回退默认路由

5. 各阶段的具体安排

P0：冻结设计总线

当前状态

设计总线、详细测试规格和 LangGraph 迁移计划均已按当前版本决策同步；P0 已完成。
P1 和 P2 也已在后续阶段完成，项目下一步进入 P3。

产出

- 三大功能的设计意义；
- Transaction 与 Scene 的职责边界；
- Transaction 四状态及主要状态变化；
- conversation 内刺激池的职责；
- 有来源与无来源刺激的分流逻辑；
- 三大功能的端到端领域主线；
- activation、UI Pause/Delete、Feedback 因果校验、一次性 Schedule 和异步工具边界；
- 旧迁移计划与新详细设计中的状态权威边界得到统一。

退出条件

- 三大功能边界获得确认；
- 详细测试规格已经同步当前版领域语义；
- 不再把幂等、恢复、Thinking、工具审计等横向约束提升为第四个结构性功能；
- 后续替换框架、存储或类结构时，不需要改变这份设计总线。

- 旧迁移计划的状态权威和阶段入口已经修订，P1 不再同时面对两套冲突方案。

P1：冻结跨 Runtime 测试契约

当前状态

P1 已完整实现并满足退出条件：

- ScenarioSpec/ScenarioVariantSpec/RuntimeBindingSpec、校验规则和显式可用性状态均已冻结；
- 28 个顶层场景已经展开为 27 个 Core、7 个必测 Robustness 和 1 个 Matcher Evaluation，共 35 个当前必测 variant；
- think_life_v1 的 35 个 variant 全部可执行；P1 当时的结果为 35 个已登记 Known Gap、0 个未登记失败，该数字已冻结为 pre-P2 历史基线；
- langgraph_v1 的 35 个对应 binding 全部明确为 Not Implemented；
- 共享 Runtime Harness、ThinkLife adapter、共享断言和 normalized observation/trace 已覆盖完整矩阵；
- AT-10 已冻结 30 条 matcher 评测数据和独立 evaluator；
- 详细规格中的 25 条 Robustness requirement 已全部映射到 7 个必测 variant；
- Scenario CLI、Web 分层工作区、结构化报告和 ThinkLife P1 分层差距表均已完成；
- 旧 INV-01~INV-14 继续作为 Supporting Tests 和历史映射。

P1 的完成标准是证据覆盖完整且状态诚实，不是 Runtime 全绿。P3 后已有 20 个完整 variant 转绿，当前剩余 15 个已登记差距由 P3 ~ P5/P7 继续消除；M0 仍未通过。

目标

先让测试准确表达目标语义，再修改 Runtime。

已完成内容

1. 已把 27 个确定性 TX/SP/AT Core 场景转换为数据化、可执行的共享场景；AT-10 作为独立 Matcher Evaluation。
2. 已为第 8 节当前版本横向约束建立 parent_scenario_id + variant_id + layer=robustness manifest，并至少实现测试规格第 9 节阻断清单中的共享场景；具体断言来自测试规格第 8 节与第 10.3 节，包括 UoW replay、Schedule 原子控制、durable Inbox/lease fencing、result→Feedback outbox、三种 capability guarantee 与 Flush fault。
3. 已定义统一 Runtime Harness，操作与观察面包括：
 - 创建 conversation；
 - 提交刺激；
 - 查询刺激池；
 - 查询、创建和恢复 transaction；
 - 观察 pause reason、lifecycle、activation、delegate 和 transaction revision；
 - 对单个 transaction 执行 UI Pause/Delete；
 - 注入可控异步工具并观察 Feedback admission/preconsume disposition；
 - 观察 stimulus ingress key、accepted_seq、conversation consumer owner/lease、consumer/claim epoch、StaleClaim 与最终 disposition；

- 观察 schedule ID、run ID、delivery ID/generation、run status、claimed activation、blocked reason 与目标绑定；
- 观察 effect result、Feedback ingress outbox、capability guarantee、attempts、ack/result、visible-effect count/range 与 uncertain；
- 读取 Scene；
- 触发 flush；
- 注入 fake matcher、fake effect sink 和故障点；
- 导出 normalized semantic trace。

4. 已实现 `think_life_v1` adapter。
5. 已为尚不存在的 `langgraph_v1` 明确返回 `Not Implemented`。
6. 已将 UI 组织为 TX/SP/AT → 测试层级 → Runtime → 场景证据。
7. 已将旧 INV 用例定位为 Supporting Tests 和历史映射。
8. 已对 ThinkLife 执行 27 个新 Core、必测 Robustness manifest 和 `AT-10`，生成第一份分层完整差距表。

退出条件（已满足）

- 详细测试规格不再包含“Scheduled Plan 执行后回到 pause”或“只凭 transaction ID 接受 Feedback”等旧语义；
- 27 个 Core 场景不引用具体 Runtime 的类、私有方法或节点；
- 每个 Core 场景可以参数化选择 Runtime backend；
- 每个 variant 都有明确 parent ID 和 layer；当前版本 Core 与必测 Robustness 的输入及断言均已存在，不得仍为 `Not Covered`；
- `AT-10` 使用冻结的 matcher 输入输出契约和独立数据集；
- `think_life_v1` 的 35 个当前必测 variant 均可实际运行；
- 未实现的 `langgraph_v1` 在 35 个对应 binding 上均明确显示 `Not Implemented`；
- 非当前必测范围的缺测项目仍明确显示 `Not Covered`，但不能用该状态让 P1 跳过当前必测契约；
- normalized trace schema 冻结；
- Harness 提供 admission/preconsume disposition、唯一 ingress/claim 和异步工具消费边界的统一操作与观察面；当前 Runtime 缺失的目标事实以明确 check 证据形成 Known Gap；
- ThinkLife 的 Core/Robustness/Matcher 分层差距清单已经生成。

P1 退出时的历史结果为：当前必测项 0 `Not Covered`，ThinkLife 35 个可执行、35 个 Known Gap、0 个未登记失败，LangGraph 35 个 `Not Implemented`。P3 后复验已变为 20 Passed / 15 Registered Known Gap / 0 Unexpected；两组数字分别服务于历史基线和当前状态，不能混用。

主要风险与控制

- 风险：adapter 隐藏实现差异。
控制：以 Store 快照和 fake effect sink 为主要证据。
- 风险：测试复制当前 Runtime 的内部结构。
控制：场景只使用领域命令和领域观察值。

P2: 实现 TX——Transaction 与 Scene 基础

当前状态

P2 TX Foundation 已完成并通过独立 Gate。实现保持在 runtime/domain/store、共享 Harness 和 fake/logical port 层，没有修改 M-Agent-UI 或 M-Agent-Desktop，也没有提前实现依赖 TX 的 AT 主链。

目标

建立 AT 所依赖、并且端到端链路需要的事务领域基础。SP 可以继续用 fake consumer 独立实施，不依赖 TX 内部运行。

已完成内容

- 目标四状态及合法转换；
- 稳定 transaction ID、每次独立激活的 activation 身份，以及 activation 内的 delegate 身份；
- WM、任务状态和 revision；
- 定义 RuntimeUnitOfWork 契约与保存 `command_digest + result payload/ref` 的 transition ledger，以 fake Inbox/Schedule ports 验证 transaction-side 原语；本阶段不冒充生产耐久跨 Store UoW；
- 在 transaction ID 已知或已经预绑定的前提下，提供 pause、complete、archive 的 Store 加载与状态机恢复原语；
- 提供 `archive → continue` 并开启新 activation 的状态转换原语，但不在本阶段实现刺激来源分流；
- 提供面向未来 UI 控制的 transaction 级 Pause/Delete 领域原语：Pause 使当前 activation 失效并进入人工 `pause`，Delete 终止生命周期；本阶段不含 Chat API 或前端接线；
- 提供“当前刺激被用户中止且不重入”的事务侧结果，以及旧 activation 和未决 delegate 的永久失效标记；
- conversation 单一 Scene；
- 多 transaction 隔离；
- 为已预绑定 transaction ID 的 Schedule 与 Feedback 提供事务侧导入能力；一次性 Scheduled Plan 到期时开启新 activation，执行结束进入 `complete`；
- 在契约/fake port 层提供 Schedule/Transaction/Inbox 同提交原语：
`register + scheduled_wait`、
`claim + begin activation`、`complete + consumed`、`UI Pause + blocked`，以及
`UI Delete + cancelled`；
- 正常 flush 与 archive 关系；
- Transaction、Scene 的参考持久化，以及 Inbox、Schedule、Effect Ledger、Flush Journal 的领域 port；生产耐久共享实现放到 P7。

关键实现位于：

- [`contracts.py`](#)
- [`transaction/domain.py`](#)
- [`transaction/store.py`](#)
- [`transaction/uow.py`](#)
- [`transaction/schedule.py`](#)
- [`transaction/flush.py`](#)

- [transaction_registry.py](#)
- [test think life tx foundation.py](#)

延后到后续阶段

- P3: durable Inbox、admission/preconsume、conversation consumer/lease、claim/disposition 和 worker latch 已接入；in-flight stimulus abort/no-reentry 与异步 effect→Feedback 的生产级外层仍需后续收口；
- P4: source routing、候选过滤、matcher 和 TX mixed variants 的端到端收口；
- 前端阶段：Chat API 控制端点、M-Agent-UI 与 M-Agent-Desktop 的 transaction 控件；
- P7: 真实跨 Store UoW、effect ledger/outbox、外部物化和更强的进程故障测试。

退出条件

- 不依赖 SP/AT 的 TX Foundation Gate 全绿，包括 transaction 创建/加载、四状态与 activation 原语、面向 UI 控制的 Pause/Delete 领域原语、持久恢复、隔离、Scene 连续性和 flush 提交边界；
- 依赖来源分流或无来源匹配的 TX 端到端场景保持明确待办，在 P4 接入 AT 后收口，不能提前算作通过；
- 已知 transaction ID 前提下的 pause、complete、archive 加载与状态恢复原语符合规格；P2 不验证 stimulus 来源分流或 matcher；
- Pause 后旧 activation 永久失效；恢复产生新 activation，旧批次不能因 transaction 状态恢复而重新有效；
- 一次性 Scheduled Plan 的事务侧状态线最终进入 `complete`；
- 多个 transaction 的 WM 和任务状态不串；
- Scene 单调且不被 transaction 生命周期替代；
- flush 不错误归档 pause 或 continue；
- 同一类状态在任一时刻只有一个权威来源。

当前 Gate 命令：

```
python -m pytest -q -m p2_foundation
python -m pytest -q tests/acceptance/scenarios/test_tx_scenarios.py
```

第二条命令的预期不是“零 xfail”：P2 已移除 6 个真实通过 variant 的 gap；其余 mixed variant 只要仍依赖 P3/P4/P7，就必须继续显示已登记差距，且不得出现未登记失败。

主要风险与回退

- 风险：Registry、持久存储和未来 GraphState 同时成为权威。
回退：通过 adapter 或 feature flag 选择单一权威后端，禁止双写权威状态。
- 风险：旧数据无法读取。
回退：保留 schema version 和旧数据读取适配，不原地破坏旧记录。

P3：实现 SP——Conversation 内刺激池

目标

形成独立于 transaction 执行引擎的接纳和调度层。

工作内容

- 每个 conversation 独立的逻辑刺激池；
- 稳定 stimulus ID、唯一 ingress key、accepted_seq、claim 与最终 disposition；
- P3 已接入 SQLite 参考 Store-backed Stimulus Pool，并在同一 Runtime Store 内提供 partition owner/lease、consumer/claim epoch 与 StaleClaim fencing；P7 保留生产级共享外层、effect/result outbox 与跨 Runtime 接入强度；
- 多种 stimulus 及来源信息的完整保存；
- Feedback admission Gate：在入池前读取 transaction、activation 和 delegate 有效性；
- 有来源刺激的 preconsume 原子复核：排队后失效的 Feedback 无副作用终止，不进入 AT；
- 有效优先级与同优先级接纳序号；
- 严格逐个消费 Thinking 刺激及其状态提交；
- 每 conversation 单消费者；
- Thinking 发出工具委托后立即结束本次刺激消费，工具异步执行并以后续 Feedback 再次申请入池；
- UI Pause/Delete 通过直接控制面生效，不排入普通刺激池；
- 多生产者并发接纳；
- consumer 退出窗口的可靠唤醒；
- schedule 到期后才生成并入池；
- Schedule 在 preconsume 进入 blocked/cancelled/idempotent 分支时，delivery 以带原因的 handled 终结，不留在 claimed，也不伪装成 Expected Discard；
- 已归因刺激重入时保持身份和归属。

当前接入状态

- StimulusEnvelope 已携带 accepted/disposition/claim 元数据；
- SQLiteRuntimeStore 已新增 stimulus partition 与 stimuli 表；
- StimulusInbox 在生产 runtime 与 harness 中已切换为 Store-backed ready/claim 查询；
- PerceptionGateway 已执行 priority range 校验、ingress 去重与 Feedback admission discard；
- preconsume 复核失败会写入 expected_discard 的 stage/reason；
- Harness 已支持 list/load partition、acquire/takeover、claim/finalize、worker latch 与 disposition 等 P3 观察面；
- 已归因 stimulus 重入优先使用显式 transaction_id，避免重复 matcher 或新建 transaction。
- 最新全层 Gate run_37f4ad28ec8f45158e08 为 20 Passed、15 Known Gap、0 Unexpected；11 个 stale Known Gap 已从 catalog 和测试装饰器中清理。

退出条件

- ThinkLife 的 SP Core Gate 全绿；
- 不同 conversation 不互取刺激；
- 优先级和 FIFO 行为确定；
- 同一 delegate 的并发重复 Feedback 只能形成一个规范 stimulus；
- lost-wakeup 场景通过；
- 人工暂停、删除、旧 activation 和错误 delegate 的 Feedback 在 admission 或 preconsume 的正确阶段得到 Expected Discard；

- 未开启 activation 的 Schedule delivery 得到 handled 终态，并与 run 状态一致；
- fake tool 尚未返回时，下一刺激已经可以进入 Thinking，证明工具生命周期未占用消费者；
- 并发提交的数量、内容和顺序可解释；
- 池本身不进行 transaction 匹配或创建。

主要风险与回退

- 风险：同一刺激被两个消费者取得，或者刺激到达后无人唤醒。
回退：按 conversation 固定池版本，任何时刻只允许一个 drainer。
- 风险：把异步工具执行错误地包含在一次刺激消费中，造成 conversation 队首阻塞。
控制：以 Thinking 完成状态提交和发出委托作为消费边界，工具结果只通过新 Feedback 返回。
- 风险：切换时旧池和新池同时消费。
回退：切换前排空，或者显式迁移待处理刺激，不能双池并行取同一 conversation。

P4：实现 AT——来源分流与语义归因

当前状态

P4 已完成 ThinkLife AT Core Gate：AT-01~AT-09 全绿，TX-03 未 Flush Complete 候选与显式恢复已收口。P5 已继续把 AT-10 阈值、TX-06/07 Core、SP-06 与 lease takeover 收口到 M0；剩余 outbox / capability / Schedule 控制竞态归 P7。

目标

在 TX 和 SP 稳定以后实现 transaction 归因。

工作内容

- 有明确 transaction ID 时不调用语义 matcher，但先执行来源类型对应的确定性有效性校验；
- 合法显式 UI/控制恢复可以定位 pause、未 flush complete 或 archive，并开启新 activation；
- Feedback 同时校验 transaction_id + activation_id + delegate_id，且只能继续当前有效执行批次；
- 人工暂停、删除、旧 activation 或无效 delegate 的 Feedback 明确废弃，不能恢复 transaction 或新建 transaction；
- ID 不存在时明确拒绝；
- archive 由合法显式动作按明确 ID，或由有效预登记的一次性 Schedule run 恢复并开启新 activation；自然语言 archive 检索留待后续版本；
- 无来源刺激只查询当前 conversation 的 pause 和未 flush complete；
- 无来源刺激匹配 pause 后，保留原 transaction 身份但开启新 activation；该路径不受 Feedback admission 拒绝规则阻止；
- 无来源刺激匹配 pause(awaiting_feedback) 时，在一个 revision 转换中失效旧 activation/delegate 并开启新 activation；
- fake matcher 驱动确定性 Core Gate；
- 无匹配、模糊、异常或非法候选统一新建；
- 真实 Matcher v1 使用独立合成样本评估。

退出条件

- ThinkLife 的 AT Core Gate 全绿；

- P2 留待归因接入的 TX 端到端场景完成收口，尤其是 TX-02、TX-03、TX-05、TX-06 和 TX-07 的来源直达、匹配、恢复与状态转换；
- 候选集合越权数量为零；
- 所有有来源刺激调用 matcher 次数为零，但无效来源不会因此绕过确定性校验；
- Transaction 恢复到正常状态以后，旧 activation 晚到 Feedback 仍然保持 Expected Discard；
- deleted transaction 不进入候选集，也不能被任何 Feedback 恢复；
- 模糊判断不会污染已有 transaction；
- 新建与恢复的状态变化正确；
- 真实 Matcher 达到测试规格规定的首版阈值。

主要风险与回退

- 最高风险是错误复用 transaction，污染已有 WM。
回退：无来源 matcher 可以切换到“一律新建”的安全模式。
- 真实模型存在波动。
控制：真实模型评测独立于确定性 Core Gate。

P5：完成 M0 基线

当前状态

P5/M0 已完成复验：run_0036c18868874e58a39e 显示 27 Core 全绿与 3 个待收口 Robustness 差距（P7 已转绿）。AT-10 独立达标。

目标

把三类功能接成完整链路：

```
Feedback 来源有效性校验 / 普通 stimulus 接纳
→ 合法 stimulus 入池
→ 按 conversation 内优先级逐个取出
→ 有来源确定性校验后直达 / 无来源匹配或新建
→ Thinking 推进 transaction 并按需发出异步工具委托
→ 不等待工具完成，继续选择下一 stimulus
→ 工具结果作为 Feedback 重新经过 admission
→ Scene 记录
→ 持久化或 flush
```

退出条件

- think_life_v1 的 27 个 TX/SP/AT Core Gate 全绿；
- AT-10 Matcher Evaluation 独立达到规格阈值；
- Robustness 中尚未完成的项目都有明确状态；
- UI 可以按三大类、Runtime 和测试层级展示；
- 暂停—恢复—旧 Feedback 晚到竞态通过，旧批次不能污染新 activation；
- 异步工具尚未完成时，后续刺激仍可正常消费；
- Expected Discard 在报告中有明确原因，但场景本身按预期通过；

- 没有 Not Covered 或 Not Implemented 被算作通过；
- 旧 6 个 Known Gap 已修复，或者已被新规格明确替代并说明原因。

M0 已通过； P6 LangGraph transaction graph PoC 已完成。

P6: LangGraph Transaction Graph PoC

当前状态

已完成。实现位于 `src/m_agent/runtime/langgraph/`，验收 adapter 为 `langgraph_v1`。 `langgraph_v1` 已在以下 5 个 variant 上可执行并与 ThinkLife 产出可比较的 normalized semantic trace：

Variant	Layer	验证重点
TX-01/core	core	单已归因 transaction 的 WM/任务状态/commit； GraphState 不替代 Store 权威
TX-01/uow_replay_and_result	robustness	UoW 已提交/checkpoint 未推进窗口； transition 重放
TX-01/poc_checkpoint_resume	poc	pause→continue； 新 activation； WM/任务状态保留
TX-01/poc_sequential_fake_effects	poc	顺序 fake capability + <code>reply_to_user</code> ； 独立 delegate/effect
TX-01/poc_stale_feedback_after_restore	poc	新 activation 后旧 Feedback Expected Discard

其余 33 个 `langgraph_v1` binding 仍明确为 Not Implemented。

目标

只验证“单个已完成归因的 transaction 是否适合由 LangGraph 表达”，不同时迁移 SP、AT、Scene、schedule 和真实副作用工具。

首个实验范围

- 一个已完成归因的用户刺激；
- 一个 transaction；
- 按 PoC 子场景顺序创建多个 activation，任一时刻最多一个有效；
- 临时 checkpoint；
- 两个顺序执行、无副作用、支持稳定 idempotency key 且可控异步完成的 fake effect：普通 capability 与 `reply_to_user`，各自拥有 delegate 和结构化 Feedback；
- `continue/pause/complete`；
- 共享 TX 场景和 normalized trace。

退出条件（已满足）

- 完整运行共享 TX-01/core，证明已完成归因的新 transaction 可以在两个 Runtime 中得到相同的身份、WM、任务状态和最终状态；
- 运行明确标记为 PoC Slice 的恢复场景：验证 `pause → continue` 和原 WM/任务状态恢复；

- 顺序运行普通 capability 与 `reply_to_user` 两次异步 fake effect; 每次创建独立 delegate 与结构化 Feedback;
- 故障注入 UoW 已提交但 checkpointer 尚未推进的窗口; 同一 transition 重放返回第一次完整 ID/结果;
- 验证 transaction 暂停并以新 activation 恢复后, 旧 activation 的 fake Feedback 被拒绝;
- 两个 Runtime 对上述切片产生可比较的 normalized semantic trace;
- `langgraph_v1` 的其余 TX/SP/AT 场景继续明确显示 `Not Implemented`;
- checkpoint 不替代 Scene; GraphState 与 Transaction 权威边界明确; 图结构能够自然表达 transaction 语义。

回退

PoC 与生产入口完全隔离。如果图结构不能简化运行模型, 或者必须改变领域语义才能使用, 则重新评估图边界, 不影响旧 Runtime。

P7: 持久恢复与三大功能接入

建议顺序

1. Transaction graph checkpoint 和 resume;
2. 耐久 Stimulus Inbox、唯一 ingress、partition owner/lease、consumer/claim epoch fencing、accepted_seq 与 disposition 恢复;
3. effect ledger、capability guarantee、result→Feedback outbox 与稳定幂等键;
4. Schedule/Transaction/Inbox 的统一 UoW;
5. 将耐久 Inbox/UoW 作为 ThinkLife 与 LangGraph 共用外层接入;
6. 接受 SP 选出的已到达刺激;
7. 接受 AT 已确定的 transaction;
8. 根据 transaction 固定的 Runtime 归属运行或恢复;
9. 映射 Scene 和 normalized runtime events;
10. 对两个 Runtime 运行完整 Core、Store conformance 与必测 Robustness。

退出条件

- 共享耐久 Inbox/UoW 接入后, `think_life_v1` 与 `langgraph_v1` 都重新运行 27 个 Core Gate; 两者的 SP Core、Store conformance 与 Inbox/UoW 必测 Robustness 全绿;
- 重启不会丢失 WM、任务状态或 transaction ID;
- 重启不会丢失已接纳 stimulus、accepted_seq、claim/disposition 或 effect result;
- Feedback outbox 对 accepted、idempotent existing 与 admission Expected Discard terminal ack; 只有基础设施失败重试, 不热循环 stale Feedback;
- lease takeover 后旧 claimant 的新 transition 得到 `StaleClaim` 且零 mutation/effect; takeover 前已提交但未响应的相同 ID/digest 重放返回原结果;
- 节点重放和重复 Feedback 不重复推进 transaction; 支持幂等 key 的 capability 不产生第二次可观察外部效果, 非幂等 capability 按声明的 `at_most_once / at_least_once` guarantee 暴露 `uncertain`, 不能伪报 `exactly-once`;
- guarantee conformance 证明: `idempotent` 可多 attempt 但可见效果一次; `at_most_once` attempts 等于 1 且失联不重试; `at_least_once` 在 destination

恢复可达时以同 key 重试至确认并至少产生一次效果；

- 多 transaction 不串扰；
- 一次性 Scheduled Plan 激活原 transaction、开启新 activation，并在本次执行完成后进入 `complete`；claimed run 在 UI Pause/Delete 竞态中分别原子进入 `blocked/cancelled`，被新 activation supersede 时解除旧绑定，awaiting_user 恢复时重新绑定；多 run 不并行 claim，delivery 的 consumed/handled/aborted disposition 不悬挂或倒改；
- 人工暂停、删除、旧 activation 和无效 delegate 的 Feedback 在两个 Runtime 中得到相同的 Expected Discard；
- archive 可按 ID 恢复；
- Scene 与 flush 行为和 ThinkLife 基线一致。

回退

- transaction 创建时固定其 Runtime 归属；
- 已开始运行的 transaction 不在中途切换 Runtime；
- 新 Runtime 出现问题时，只把后续新建 transaction 的默认路由切回旧 Runtime；
- effect ledger、result outbox 和 capability guarantee 未通过前不开放真实副作用工具。

P8：灰度与旧 Runtime 收缩

当前状态

P8 已完成。实现摘要：

- **持久 checkpoint**： `src/m_agent/runtime/langgraph/checkpointer.py` 使用 `langgraph-checkpoint-sqlite`；Harness 在 `{persist_root}/langgraph-checkpoints.sqlite3` 持久化 graph thread 状态，与 Transaction Store 分离。
- **完整矩阵**： `langgraph_v1` adapter 覆盖 TX/SP/AT 全部 38 variant；TX-01 保留 graph 原生 handler，其余通过共享 Store 外层与 ThinkLife 等价 handler 驱动。
- **灰度路由**： `src/m_agent/runtime/routing.py`；transaction 创建时固定 `runtime_engine`；`M_AGENT_DEFAULT_RUNTIME_ENGINE` 控制新 transaction 默认路由，回退只影响后续新建 transaction。
- **验收**： `M_AGENT_ACCEPTANCE_RUNTIME_ID=langgraph_v1` 下 38 variant 全绿。

创建 transaction 时固定：

```
runtime_engine = "think_life_v1" | "langgraph_v1"
```

灰度顺序

1. fake tool；
2. 内部 conversation；
3. 新建的普通用户 transaction；
4. Scheduled Plan transaction；
5. 默认启用 LangGraph；
6. 稳定观察期以后再删除旧 transaction 循环。

退出条件（已满足）

- 双 Runtime 完整矩阵可以比较；
- 灰度期间没有状态串扰、重复效果或 Scene/flush 漂移（共享 Store 权威边界保持）；
- checkpoint schema 使用独立 SQLite 文件，与 Transaction Store 分离；
- runtime_engine 在 transaction 创建时固定并可耐久保存；
- 回退只影响后续新 transaction（M_AGENT_DEFAULT_RUNTIME_ENGINE 切回 think_life_v1）。

尚未完成（稳定观察期后再做）：

- 不再存在由旧 Runtime 管理的非终态 transaction；
- 默认路由切到 LangGraph 的生产观察；
- 删除旧 transaction 内循环。

回退

- 只改变“新 transaction”的默认 Runtime；
- 已有 transaction 继续由创建它的 Runtime 完成；
- runtime_engine 与 engine schema version 随 transaction 全生命周期保存；若迁移 archive，必须在无活跃 activation 时显式转换并保留审计，不能静默切换；
- shadow 模式只比较 planning、state 和 trace，不能让两个 Runtime 同时执行有副作用的工具。

6. 当前立即执行的下一步

P8 灰度与 LangGraph 完整矩阵已完成。下一步按以下顺序推进：

1. 稳定观察期：在 fake tool 与内部 conversation 环境持续运行双 Runtime 对比，确认无状态串扰、重复效果或 Scene/flush 漂移。
2. 逐步把新建 transaction 默认路由切到 LangGraph（设置 M_AGENT_DEFAULT_RUNTIME_ENGINE=langgraph_v1），保留一键回退到 ThinkLife。
3. 观察期通过后，清点非终态 transaction 与可恢复 archive，再安全收缩旧 transaction 循环。
4. 视需要并行接线 Chat API / 前端 transaction 控件，但这不是 Runtime Gate 的前置条件。

7. 文档之间的关系

文档	作用	稳定程度
overall-design-architecture.md	定义三大功能为什么存在、各自负责什么、领域主线如何运行	最稳定；实现变化不应轻易修改
runtime-migration-semantic-test-spec.zh-CN.md	把领域主线展开为可验收场景，并记录首版详细方案	当前版已同步；P1 已将当前必测项全部转换为可执行契约
current-project-progress-and-design-plan.md	记录当前完成度、缺口、实施顺序和决策门	按阶段持续更新
langgraph-runtime-migration-plan.zh-CN.md	记录 LangGraph 迁移边界、风险和 P0 ~ P8 实施路径	P8 完整矩阵与灰度路由已完成；稳定观察与旧 Runtime 收缩为下一入口

文档	作用	稳定程度
semantic-acceptance-platform.zh-CN.md	说明当前验收平台的使用方法、P1 合同状态和旧 Supporting 目录	已同步完整 P1 Harness、35 个 variant、CLI/Web/report 和 UI 分层工作区
think-life-p1-gap-matrix.zh-CN.md	记录 ThinkLife 对 P1 全矩阵的分层执行结果和已登记差距	P1 基线：35 可执行、35 Known Gap、0 未登记失败

如果这些文档出现冲突，优先级应为：

已确认的设计总线

→ 当前目标语义基线（P1 后由共享可执行契约承接）

→ 当前阶段计划

→ Runtime 的现有实现细节